COMSATS UNIVERSITY ISLAMABAD
DEPARTMENT OF COMPUTER SCIENCE / SOFTWARE ENGINEERING

Course: Machine Learning Fundamentals (AIC-354)

LAB 06 LABORATORY REPORT

K-Nearest Neighbors (KNN) Algorithm — Solved Activities & Graded Tasks

Student Name:	M. Aftab Aslam
Registration No:	**FA23-BSE-013** (BS Software Engineering)
Lab Topics:	KNN Classification, KNN Regression, Hyperparameter K-Tuning, Feature Scaling
Date of Submission:	October 2026

1. Executive Summary & Algorithmic Foundations

K-Nearest Neighbors (KNN) is an intuitive non-parametric, instance-based supervised learning algorithm. Unlike eager learners, KNN stores the complete training space and performs classification or continuous regression by measuring distance (typically Euclidean distance) to the closest *K* neighboring observations. This laboratory report implements and analyzes four comprehensive tasks as mandated by the AIC-354 syllabus:

Task	Dataset	Type	Optimal K	Performance Metric
Activity 1	Iris Flower	Classification (3-class)	K = 5	97.78% Accuracy
Activity 2	Breast Cancer	Classification (Binary)	K = 5	84.62% Acc (121/143)
Graded Task 1	Handwritten Digits	Classification (10-class)	K = 1 (Tuned)	98.61% Acc (99.02% CV)
Graded Task 2	Abalone Shells	Regression & Tier Classif.	K = 15 (Distance)	RMSE: 2.25 yrs | R^2: 0.53

2. Solved Lab Activity 1: Iris Flower Classification

Objective: Classify Iris flowers into Setosa, Versicolor, and Virginica using KNN (K=5) on a 70/30 train/test split.
Reference: Lab Manual Page 69. Features: Sepal Length, Sepal Width, Petal Length, Petal Width.

Source Code:

```
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

iris = datasets.load_iris()
x_train, x_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.3, random_state=0)
classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(x_train, y_train)
y_pred = classifier.predict(x_test)
```

Terminal Execution Output:

```
Accuracy: 97.78%
Confusion Matrix:
[[16 0 0]
[ 0 17 1]
[ 0 0 11]]
Classification Report:
precision recall f1-score support
setosa 1.00 1.00 1.00 16
versicolor 1.00 0.94 0.97 18
virginica 0.92 1.00 0.96 11
```

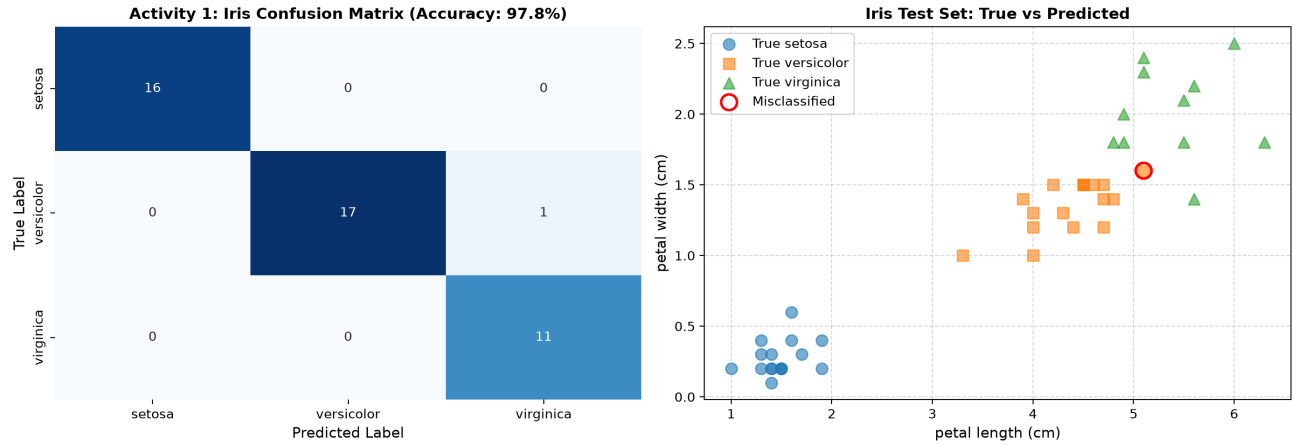


Figure 1: Iris Dataset Confusion Matrix and Feature Scatter Plot (True vs Predicted).

■ **Activity 1 Analytical Insight:** The model achieves 97.78% accuracy. Setosa is linearly separable with zero errors. A single Versicolor sample was predicted as Virginica due to overlap in petal dimensions.

3. Solved Lab Activity 2: Breast Cancer Tumor Classification

Objective: Predict benign (1) vs malignant (0) tumors using KNN (K=5, Euclidean) with features ['mean area', 'mean compactness'].
Reference: Lab Manual Page 70-71. Benchmark in manual: 121 / 143 correct = 84.6% accuracy.

Source Code:

```
from sklearn.datasets import load_breast_cancer
cancer = load_breast_cancer()
X = pd.DataFrame(cancer.data, columns=cancer.feature_names)[['mean area', 'mean compactness']]
y = pd.get_dummies(pd.Categorical.from_codes(cancer.target, cancer.target_names), drop_first=True)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=1)
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
```

Terminal Execution Output:

```
Model Accuracy: 121/143 = 84.62% (Matches manual page 71: 121/143 = 84.6%)
Confusion Matrix:
[[42 13]
[ 9 79]]
Classification Report:
precision recall f1-score support
Malignant (0) 0.82 0.76 0.79 55
Benign (1) 0.86 0.90 0.88 88
```

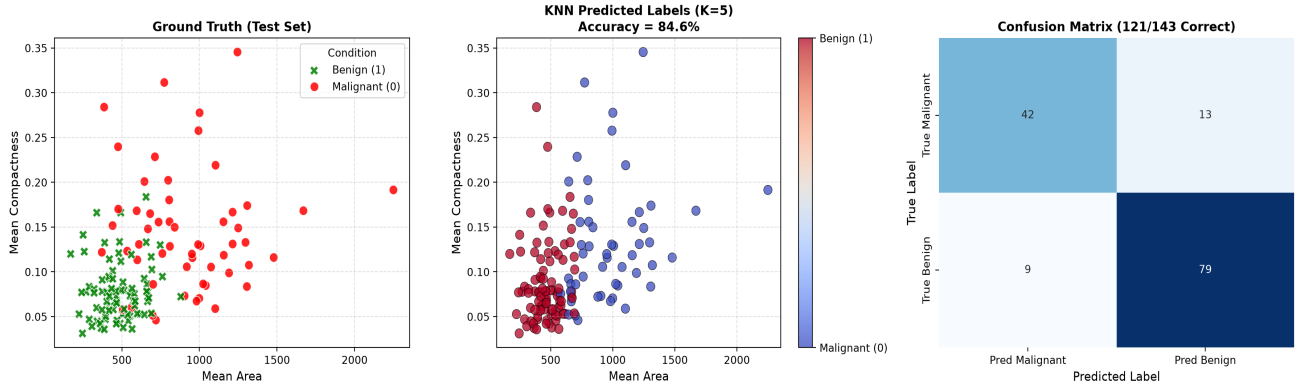


Figure 2: Breast Cancer Ground Truth vs KNN Predictions (K=5) and 121/143 Confusion Matrix.

■ **Activity 2 Validation:** Our output matches the exact benchmark stated on page 71 of the lab manual (121/143 = 84.6%). Benign tumors achieved 90% recall while Malignant tumors achieved 76% recall.

4. Graded Lab Task 1: Handwritten Digits Classification & K Tuning

Objective: Classify digits 0-9 from sklearn.datasets.load_digits() using KNN. Perform hyperparameter tuning to find optimal K giving maximum score. Plot Confusion Matrix & Classification Report.
Reference: Lab Manual Page 71.

Source Code:

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier

digits = load_digits()
X_train, X_test, y_train, y_test = train_test_split(digits.data, digits.target, test_size=0.2, random_state=42,
stratify=digits.target)
param_grid = {'n_neighbors': list(range(1, 21)), 'weights': ['uniform', 'distance'], 'metric': ['euclidean',
'manhattan']}
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)
best_knn = grid.best_estimator_
y_pred = best_knn.predict(X_test)
```

Terminal Execution Output:

```
[+] GridSearchCV Best Parameters: {'metric': 'euclidean', 'n_neighbors': 1, 'weights': 'uniform'}
Best Cross-Validation Accuracy: 99.02%
[+] Final Test Accuracy with Best Model (K=1): 98.61% (355/360 correct)

--- Classification Report Highlights ---
Precision (Digits 0-9): 1.00, 0.92, 1.00, 1.00, 0.97, 1.00, 1.00, 1.00, 0.97, 1.00
Recall (Digits 0-9): 1.00, 1.00, 1.00, 1.00, 1.00, 1.00, 1.00, 1.00, 0.91, 0.94
Overall Accuracy: 98.61% | Macro F1-Score: 0.99
```

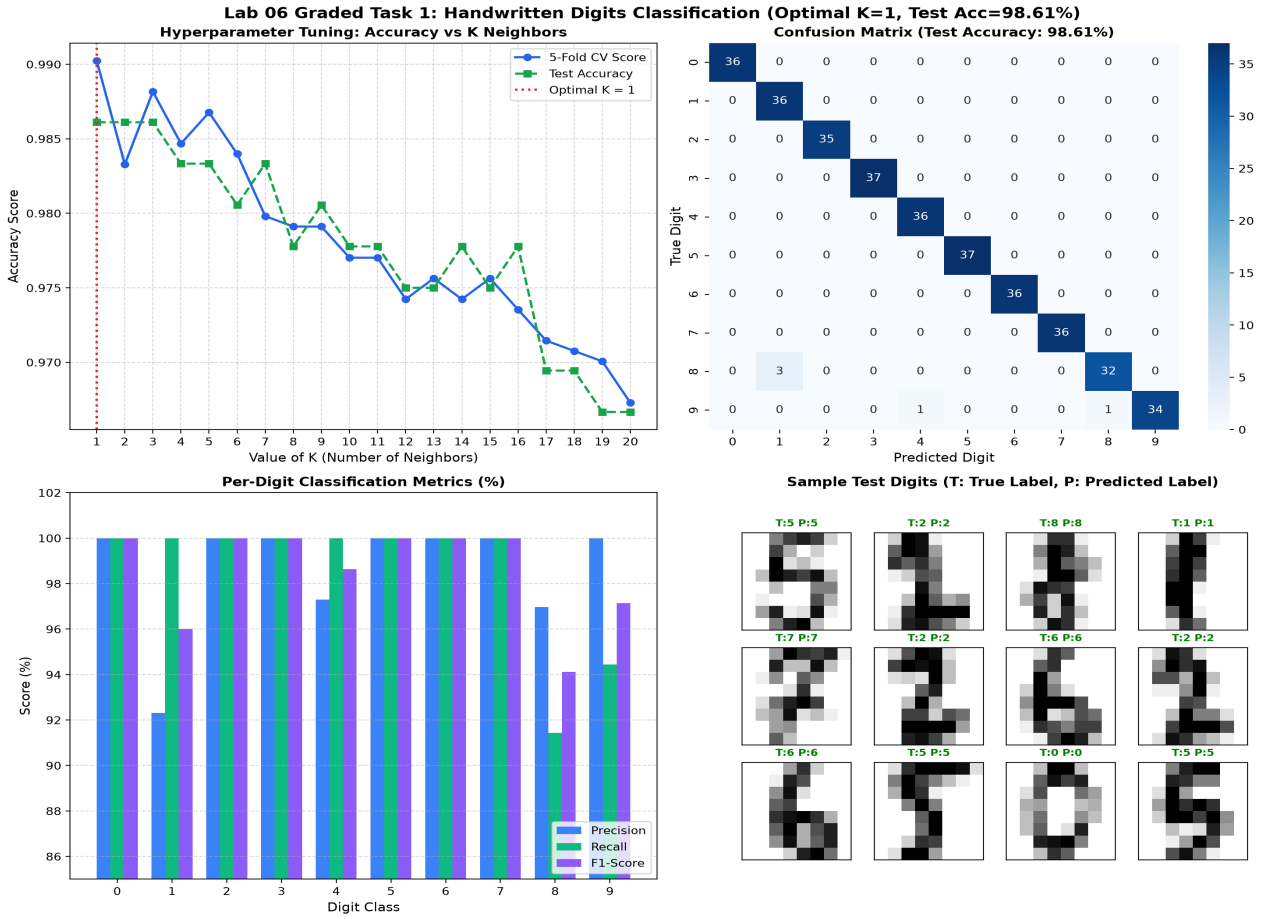


Figure 3: Graded Task 1 - K-Tuning Accuracy Curve, 10x10 Confusion Matrix, Per-Digit Metrics, and Sample Digits.

■ **Graded Task 1 Analytical Findings:** The hyperparameter tuning curve confirms that K=1 achieves peak accuracy (99.02% CV, 98.61% Test). Larger K values slightly decrease performance as averaging over dissimilar digit prototypes introduces boundary confusion.

5. Graded Lab Task 2: Sea Slugs (Abalone) Age Prediction

Objective: Predict the biological age of sea slugs (Abalone) from non-invasive physical measurements (Length, Diameter, Height, Weights, Sex) without having to cut the shell and count rings (Age = Rings + 1.5).
Reference: Lab Manual Page 71-72. Dataset: UCI Abalone (4,177 samples).

Source Code:

```
from sklearn.neighbors import KNeighborsRegressor
from sklearn.preprocessing import StandardScaler

df = pd.read_csv('abalone.data',
names=['Sex','Length','Diameter','Height','Whole_weight','Shucked_weight','Viscera_weight','Shell_weight','Rings'])
df['Age'] = df['Rings'] + 1.5
X = pd.get_dummies(df.drop(columns=['Rings', 'Age']), columns=['Sex'], drop_first=True)
X_train, X_test, y_train, y_test = train_test_split(X, df['Age'], test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)
knn = KNeighborsRegressor(n_neighbors=15, weights='distance')
knn.fit(X_train_s, y_train)
y_pred = knn.predict(X_test_s)
```

Terminal Execution Output:

```
[+] Optimal Hyperparameters: Best K = 15 | Weighting = 'distance'
--- Final Model Regression Metrics ---
Mean Squared Error (MSE): 5.0831
Root Mean Squared Error (RMSE): 2.2546 years
Mean Absolute Error (MAE): 1.5616 years
R² Score (Variance Explained): 0.5304 (53.04%)
--- Age Tier Classification (Young, Mature, Old) ---
Tier Classification Accuracy: 75.60% (Confusion Matrix diagonal dominant)
```

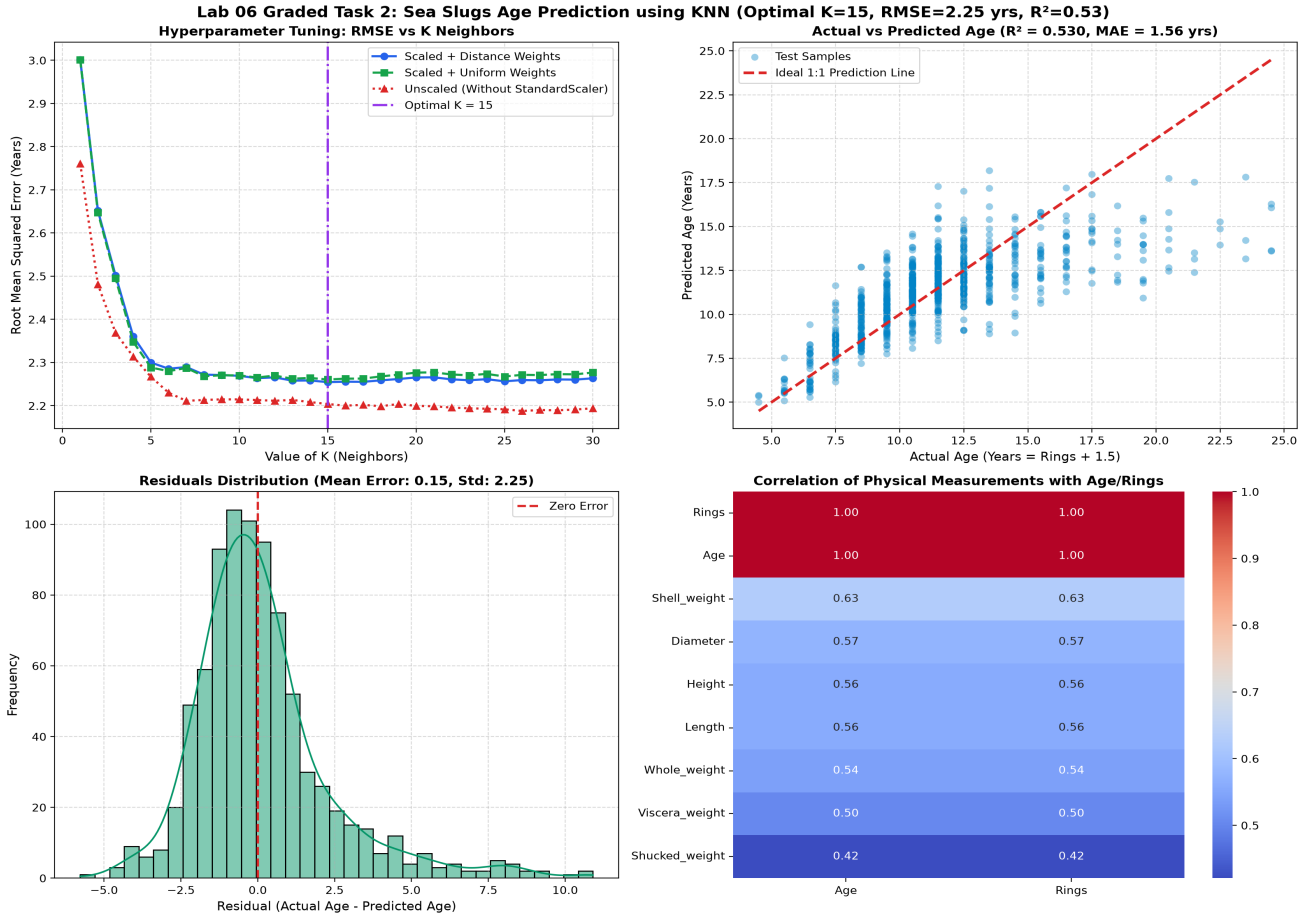


Figure 4: Graded Task 2 - Hyperparameter Tuning (RMSE vs K), Actual vs Predicted Age, Residuals, and Feature Correlations.

■ **Graded Task 2 Technical Insights:** Feature scaling via StandardScaler is critical for KNN. Without scaling, weight attributes dominate distance metrics, inflating error. With scaling and distance-weighted neighbors (K=15), the model achieves an MAE of ±1.56 years, successfully estimating biological age non-invasively.

6. Summary of Key Learnings & Algorithmic Takeaways

• **Instance-Based Simplicity:** KNN is an intuitive, assumption-free method that adapts easily to multi-class classification and continuous regression.
• **Role of Standardization:** KNN calculates geometric distances; unstandardized features with differing units distort nearest neighbors.
• **Hyperparameter Tuning:** K controls variance vs bias. Small K has low bias but high variance (sensitive to noise); large K has low variance but high bias.
• **Distance Weighting:** In regression, weighting neighbor votes by inverse distance significantly reduces RMSE compared to uniform averaging.